

刺绣定制agent-flow设计的PRD

第1章 产品概述

1.1 项目背景与目标

1.1.1 行业痛点

刺绣作为国家级非物质文化遗产，正面临“**传承难**”与“**消费难**”的双重困境。与标准化工业品不同，刺绣定制具有**强非标、重沟通、长周期**的特性——一件定制作品从需求提出到最终交付，往往要经过多轮反复确认，平均沟通成本高。

当前市场存在明显的**供给侧结构性稀缺**：尽管淘宝、Etsy等综合性平台设有手工艺品类，但**专注于刺绣细分的垂直平台几乎空白**。这导致用户不得不通过社交媒体“大海捞针”式寻找匠人，而匠人则散落在各平台被动等待订单，供需双方在信息孤岛中艰难匹配。

核心痛点

- 需求表达的“语义鸿沟”，即感性语言难以转化为技术参数

用户习惯用**氛围性词汇**描述需求：“想要温柔一点的”、“要有高级感”、“显气质但不张扬”。然而，匠人端需要的是**可执行的技术参数**：针法选择（平针/套针/打籽）、丝线规格（1/16 还是 1/32）、色彩 Pantone 色号、构图留白比例。

- 视觉确认的“失明状态”，即无样图决策导致高纠纷率

刺绣定制目前普遍处于“**盲订**”状态：用户仅凭文字描述下单，匠人凭经验想象制作。这种“**无样图确认**”模式导致最终交付时“货不对板”的纠纷频发——用户期待的是“写意水墨风”，收到的是“工笔重彩风”，双方对“淡雅”的视觉定义天差地别。

- “低效匹配”与生产过程的双盲焦虑，即供需双方难以找到合适的对方

刺绣领域的**专业能力分层**（有的匠人擅长苏绣花鸟，有的擅长粤绣猛兽）和**档期差异**。结果是：用户找不到“对的人”，匠人接到“做不了的单”，容易陷入低效循环。

此外，**进度不透明**加剧了用户焦虑——刺绣周期通常 7-15 天，用户无法知晓当前绣到哪一步，只能反复询问匠人，进一步推高沟通成本。

1.1.2 产品目标

目标：通过 4-Agent 串联降低匹配成本，实现“模糊需求 → 可执行方案”的**AI 自动化链路**

针对上述痛点，本项目旨在打造**刺绣智汇平台**——一个基于 **4-Agent AI 系统** 的垂直定制平台。核心目标包括：

- 需求标准化**：通过 Agent 1（需求解析 Agent）将用户的感性描述自动翻译为结构化技术方案（6 模块输出：构图/色彩/针法/丝线/工期/预算），将对话轮次压缩至 **≤4 轮**。
- 视觉前置化**：通过 Agent 2（图案生成 Agent）在下单前提供**可视化方案**（文字描述 + 参考图），解决“盲订”风险，实现“先确认效果，再下单制作”。
- 智能匹配**：通过 Agent 3（匹配推荐 Agent）基于复杂度等级、风格流派、档期状态进行**双向智能筛选**，可帮助降低匠人拒单率。
- 全程可追踪**：通过**进度监控工作流**提供**工序进度推送与提醒**，消除信息不对称。

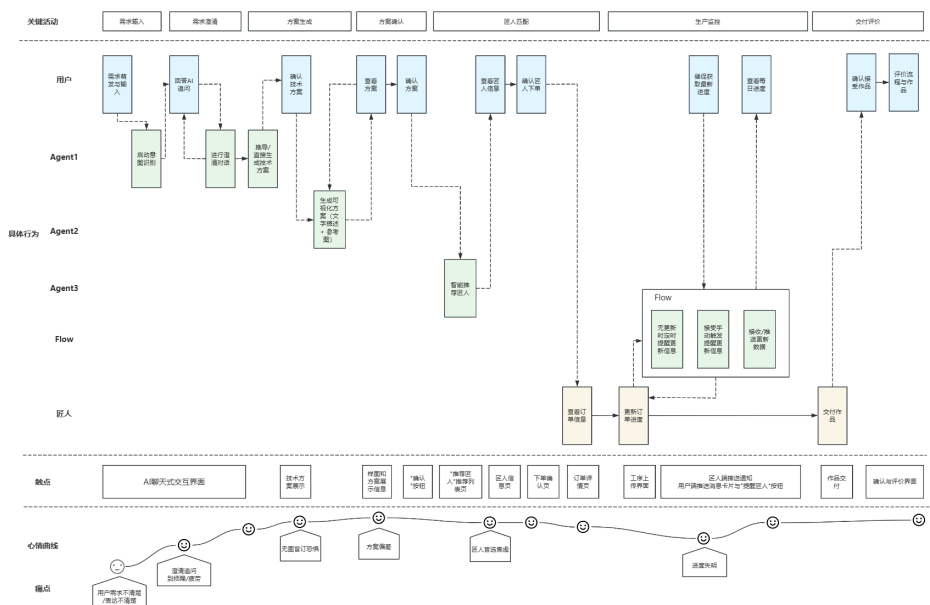
1.1.3 核心价值主张

让小白用户也能得到精准表达需求全程可见，让匠人直接拿到可开工的技术方案，通过 AI 中介降低双边交易成本，构建可持续的刺绣定制生态。

1.2 用户画像与旅程

- 核心用户：热爱刺绣，对刺绣品有独到想法的人
- 主要用户：喜欢刺绣品的人
- 合伙人用户：通过平台接受订单的匠人。

用户旅程地图：



1.3 RICE优先级排序

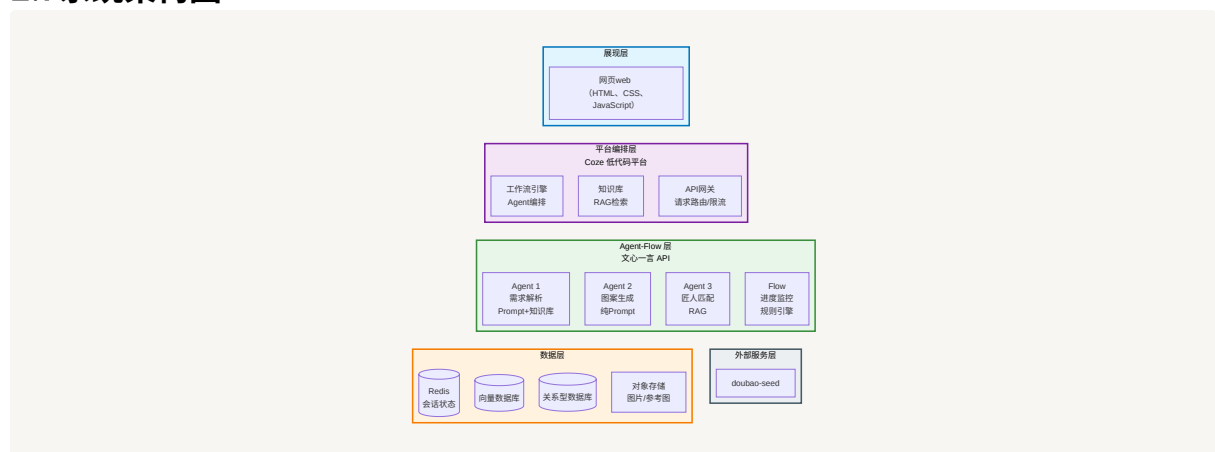
Agent	Reach	Impact	Confidence	Effort(人月)	RICE得分	排序理由
Agent 1需求解析	100%	3	90%（已验证）	0.5	$(100 \times 3 \times 0.9) / 0.5 = 540$	P0
Agent 2图案生成	100%	2	70%	0.4	$(100 \times 2 \times 0.7) / 0.4 = 350$	P1
Agent 3匹配推荐	80%	2	70%	0.5	$(80 \times 2 \times 0.7) / 0.5 = 224$	P3
Flow进度监控	60%	1.5	90%	0.3	$(60 \times 1.5 \times 0.9) / 0.3 = 270$	P2

关键洞察：

- **Flow 得分较高：**因为投入产出比最高（0.3 人月就能解决监控焦虑）。
- **Agent 3 得分偏低：**因为只有 70% 把握且需要 0.5 人月，实际优先级应低于 Flow。【可先用规则匹配 MVP】

第2章 系统架构设计

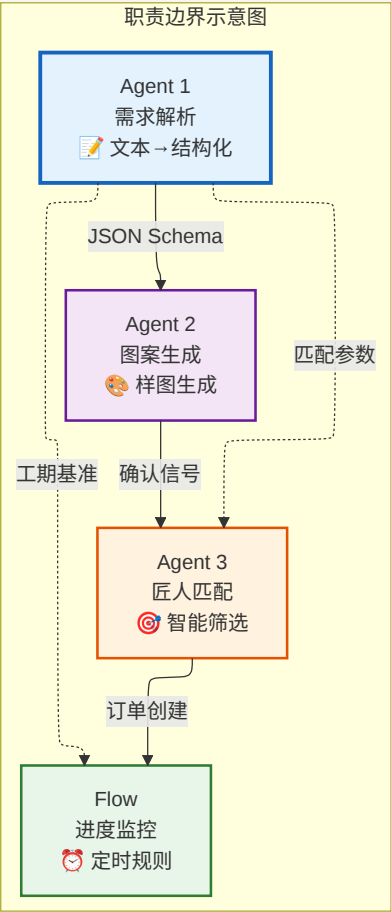
2.1 系统架构图



2.2 Agent指责边界与接口契约

2.2.1 总体职责

组件	核心职责	输入来源	输出产物	绝对不处理	异常时职责
Agent 1 需求解析	自然语言→结构化数据，需求澄清与边界拦截	用户原始文本	Agent1Output	不生成图片 不匹配匠人	识别"技术不可能"→ (识别"约束冲突"→提供三选项)/提供默认方案/人工处理
Agent 2 图案生成	结构化数据→可视化方案，创意发散与风格化	Agent1Output	图案描述方案+图案	不澄清需求 不判断预算合理性 不筛选匠人	生成失败→降级为参考图库 超时→返回文字描述，并提示等待→再次调用生成图片
Agent 3 匠人匹配	需求参数→匠人排序，双向筛选（用户需+匠人供）	Agent 2 输出（视觉方案）+ workflow上下文提供的 Agent 1 业务数据	匠人推荐 匹配度评分	不生成方案 不监控进度 不处理支付	检索为空→降级为规则匹配 无可用户匠人→等待人工介入
Flow 进度监控	工期计算→状态推送，定时检测与提醒	Agent1Output.schedule 匠人更新数据	每日进度卡片/催单通知	不解析需求 不生成方案 不参与匹配	定时失效→邮件兜底 匠人超期→双边提醒



2.2.2 跨组件异常职责划分

异常场景	识别者	处理策略	兜底方案	数据标记
技术不可能等—用户不接受建议方案	Agent 1	解释限制+提供风格化替代方案+人工介入	人工审核： 1. 评估是否接单 2. 告知用户风险 3. 特批需用户签署知情同意	全局Flow标记 <code>nextAgent='manual'</code> <code>reason='high_risk_conflict'</code> ，下游不执行

异常场景	识别者	处理策略	兜底方案	数据标记
约束冲突-用户接受建议	Agent 1	Agent 1（提供选项，按选项生成方案）	用户自选	全局Flow标记 <code>confirmationStatus='confirmed'</code> ， <code>nextAgent='agent2'</code> ，触发Agent 2
用户不确认技术方案	Flow	Flow发送提醒	48小时后自动关闭会话，标记"流失"	全局Flow标记 <code>confirmationStatus='expired'</code> 和 <code>sessionClosed=true</code>
用户要求修改技术方案	Agent 1	Agent 1局部澄清	修改超过3次 - 转人工（防止无限循环）	全局Flow检查 <code>modificationCount < 3</code> : - 是: <code>modificationCount+1</code> ，重调Agent 1 - 否: 标记 <code>nextAgent='manual'</code> ， <code>reason='excessive_modification'</code>
AI生成失败（API超时/内容违规/算力不足）	Agent 2	知识库检索（按图案+风格+复杂度匹配参考图）	返回3张参考图+ 差异说明+ "接受/转人工"选项	标记 <code>fallbackInfo.level: 1</code> ， <code>fallbackInfo.reason: 'generation_failed'</code>
参考图不匹配（相似度<0.75或用户拒绝）	Agent 2	返回风格化示意图（通用刺绣风格示意）	示意图 + 强制人工介入	标记 <code>fallbackInfo.level: 2</code> ， <code>fallbackInfo.reason: 'mismatch_rejected'</code> ， <code>nextAgent: 'manual'</code>
匹配为空	Agent 3	Agent 3降级为规则匹配	人工介入派单	记录 <code>searchMetadata</code> 供优化
定时失效	Flow	Flow（邮件兜底）	次日补发	日志记录，不影响主数据
用户投诉	业务系统	人工介入	-	读取全链路 <code>sessionId</code> 追溯

2.3 workflow

全局主流程：

用户输入 → Agent 1（结构化） → Agent 2（生成方案） → 用户选择 → Agent 3（匹配匠人） → 订单创建 → flow（每日进度）

▼ Mermaid:

```

flowchart TB
    %% ===== 样式定义 =====
    classDef startEnd fill:#e8f5e9,stroke:#2e7d32,stroke-width:2px
    classDef user fill:#e3f2fd,stroke:#1565c0,stroke-width:2px
    classDef agent1 fill:#e8f5e9,stroke:#388e3c,stroke-width:2px
    classDef agent2 fill:#f3e5f5,stroke:#6a1b9a,stroke-width:2px
    classDef agent3 fill:#fff3e0,stroke:#e65100,stroke-width:2px
    classDef flow fill:#fce4ec,stroke:#c2185b,stroke-width:2px
    classDef manual fill:#ffebee,stroke:#c62828,stroke-width:3px,color:#c62828
    classDef decision fill:#fff3e0,stroke:#e65100,stroke-width:2px,shape:stadium
    classDef fallback fill:#ecef1f,stroke:#455a64,stroke-width:2px,stroke-dasharray:
5 5
    classDef selection fill:#e1f5fe,stroke:#0277bd,stroke-width:2px

    %% ===== 主流程起点 =====
    Start([用户输入需求]):::startEnd --> A1[Agent 1<br/>需求解析+生成<br/>技术方案<br/>输出:6模块+JSON]:::agent1

    %% ===== Agent 1 阶段: Flow层状态判断 =====
    A1 --> Flow_Check{Flow层检查<br/>confirmationStatus}:::flow

    %% 分支1: 技术不可能/高风险冲突
    Flow_Check -->|nextAgent=<br/>'manual'<br/>reason='high_<br/>risk_conflict'| Manual1[人工客服介入<br/>评估/驳回/特批]:::manual
    Manual1 -->|特批通过| A1_Resume[重新生成方案<br/>标记高风险]:::agent1
    Manual1 -->|驳回| End1([结束会话<br/>reason='rejected']):::startEnd

    %% 分支2: 正常流转 → 等待用户确认
    Flow_Check -->|nextAgent=<br/>'agent2'| Flow_Pending[状态:pending<br/>等待确认技术方案<br/>confirmationDeadline=24h]:::decision

    %% 统一入口
    A1_Resume --> Flow_Pending

    %% ===== Agent 1阶段超时处理 =====
    Flow_Pending -->|24h超时<br/>confirmationStatus=<br/>'expired'| Flow_A1_Timeout[F
low发送提醒<br/>标记sessionClosed]:::flow
    Flow_A1_Timeout --> End3([会话关闭]):::startEnd

    %% ===== 用户操作分支 (Agent 1后) =====
    Flow_Pending --> Flow_UserAction{用户操作?}:::user

    %% 2.1 放弃
    Flow_UserAction -->|放弃<br/>confirmationStatus=<br/>'abandoned'| End1

    %% 2.2 修改 → Agent 1局部澄清
    Flow_UserAction -->|修改<br/>confirmationStatus=<br/>'modifying'| A1_Modify[Agent
1<br/>局部澄清<br/>保留未改字段]:::agent1
    A1_Modify --> Flow_ModCheck{Flow层检查<br/>modificationCount?}:::flow
    Flow_ModCheck -->|<3次| Flow_Pending
    Flow_ModCheck -->|≥3次| Manual_A1[转人工<br/>防止无限循环<br/>reason='excessive_modi
fication']:::manual
    Manual_A1 -->|人工整理需求| A2

```

```

%% 2.3 确认 → 进入Agent 2
Flow_UserAction -->|确认<br/>confirmationStatus=<br/>'confirmed'| A2[Agent 2<br/>视觉方案生成<br/>输入:systemData]:::agent2

%% ===== Agent 2 内部逻辑 =====
A2 --> GenCheck{AI生成<br/>结果?}:::decision

%% 2.3.1 主路径: AI生成成功
GenCheck -->|成功<br/>fallbackInfo.level=0| VisualSelect[状态:visual_pending<br/>等待用户选择<br/>24h倒计时]:::selection

%% 用户选择阶段超时
VisualSelect -->|24h无选择<br/>confirmationStatus=<br/>'expired'| Flow_A2_Timeout
[Flow发送提醒<br/>标记sessionClosed]:::flow
Flow_A2_Timeout --> End3

%% Agent 2修改循环
VisualSelect -->|选择schemeId<br/>更新selectedScheme| A3_Prep[标记<br/>isReference=false<br/>matchConfidence=1.0]:::agent2
VisualSelect -->|要求修改<br/>且modificationCount<2| A2_Modify[Agent 2<br/>重新生成<br/>modificationCount+1]:::agent2
VisualSelect -->|修改次数≥2<br/>防无限生成| Manual_A2[转人工设计师<br/>reason='excessive_visual_modification']:::manual

A2_Modify --> GenCheck
Manual_A2 -->|人工出图<br/>更新Agent2Output| A3_Prep_Manual[标记<br/>isReference=true<br/>fallbackInfo.level=2]:::agent2

%% 2.3.2 一级降级: AI生成失败
GenCheck -->|失败/超时<br/>fallbackInfo.level=1| Fallback1[知识库检索<br/>匹配参考图<br/>计算相似度]:::fallback
Fallback1 --> RefCheck{相似度<br/>≥0.75?}:::decision

RefCheck -->|是| RefSelect[展示3张参考图<br/>+差异说明<br/>+匹配度分数]:::selection

%% 参考图分支
RefSelect -->|接受<br/>确认参考图| A3_Prep_Ref[标记<br/>isReference=true<br/>fallbackInfo.level=1]:::agent2
RefSelect -->|要求修改<br/>且modificationCount<2| A2_Modify
RefSelect -->|拒绝/不匹配| Fallback2[二级降级<br/>fallbackInfo.userAccepted=false]:::fallback

RefCheck -->|否<br/>相似度<0.75| Fallback2

%% 2.3.3 二级降级: 强制人工
Fallback2 --> Manual2[转人工设计师<br/>1v1沟通<br/>nextAgent='manual']:::manual
Manual2 --> ManualDecision{人工判断?}:::decision
ManualDecision -->|人工出图<br/>或确认方案| A3_Prep_Manual
ManualDecision -->|需求不明确| End1

%% ===== Agent 3 匠人匹配 =====
A3_Prep --> A3[Agent 3<br/>匠人匹配<br/>输入:selectedScheme]:::agent3
A3_Prep_Ref --> A3
A3_Prep_Manual --> A3

```

```

A3 --> A3_CHECK{匹配结果?}:::decision
A3_CHECK -->|匹配为空| A3_FAIL[输出空列表<br/>nextAgent=manual]:::agent3
A3_FAIL --> MANUAL_MATCH[人工介入派单]:::manual
MANUAL_MATCH --> MANUAL_ORDER[人工创建订单]:::order

A3_CHECK -->|匹配成功| A3_OUT[输出Top 3匠人<br/>craftsman_pending]:::agent3

%% ===== 用户选择匠人阶段 =====
A3_OUT --> USER_OP{用户操作?}:::user

USER_OP -->|放弃| END_ABANDON([会话结束]):::startEnd
USER_OP -->|超时24h| FLOW_TIMEOUT[Flow发送提醒]:::flow
FLOW_TIMEOUT --> END_EXPIRED([会话关闭]):::startEnd

USER_OP -->|换一批<2次| A3_RETRY[重新匹配<br/>modificationCount+1]:::agent3
A3_RETRY --> A3_CHECK

USER_OP -->|换一批≥2次| MANUAL_RETRY[转人工<br/>防止无限循环]:::manual
MANUAL_RETRY --> MANUAL_MATCH

%% 确认选择匠人
USER_OP -->|确认选择匠人| ORDER_CREATE[订单创建<br/>状态confirmed<br/>合并三端数据]:::order

ORDER_CREATE --> ORDER_DB[(订单库<br/>存储完整数据)]:::flow
MANUAL_ORDER --> ORDER_DB

ORDER_DB --> FLOW_START[Flow进度监控<br/>规则引擎启动<br/>定时器激活]:::flow

%% ===== Flow进度监控 (R1-R6规则) =====
FLOW_START -->|每日8:00定时触发| Flow_Daily{R1每日播报<br/>计算进度百分比}:::flow
Flow_Daily -->|生成进度卡片| Notify_User[推送用户<br/>进度+今日任务]:::flow
Flow_Daily -->|生成任务提醒| Notify_Craftsman[推送匠人<br/>待完成工序]:::flow

%% 进度计算与状态判断
ORDER_DB -->|读取实际进度| Check_Progress{检查进度偏差}:::decision
Check_Progress -->|actual < planned<br/>且差值≥0.5天| R2_Delay[触发R2延期预警<br/>标记riskFlag:delay_risk]:::flow
R2_Delay --> Notify_Both[双边推送预警<br/>用户端+匠人端]:::flow

Check_Progress -->|剩余天数≤2| R4_Urgent[触发R4临近提醒<br/>提升频次]:::flow
Check_Progress -->|正常推进| Flow_Daily

%% R5长期停滞检测
ORDER_DB -->|检查lastUpdatedAt| Check_Stale{48小时无更新?}:::decision
Check_Stale -->|是| R5_Stale[触发R5长期停滞<br/>Flow_Stale_Progress]:::flow
R5_Stale -->|发送催单| Notify_Stale[双边提醒<br/>匠人端+用户端]:::flow
R5_Stale -->|标记stale状态| Update_Stale[progress.status=stale]:::flow

%% 72小时无响应升级
Check_Stale -->|72小时仍无更新| Require_Intervention[标记requiresIntervention<br/>触发人工介入]:::manual

%% 工序完成事件触发
Craftsman_Update[匠人标记工序完成<br/>上传照片]:::user -->|事件触发| R3_Complete[触发R3工序确认<br/>更新completedSteps]:::flow

```



```

R3_Complete -->|重新计算| Check_Progress

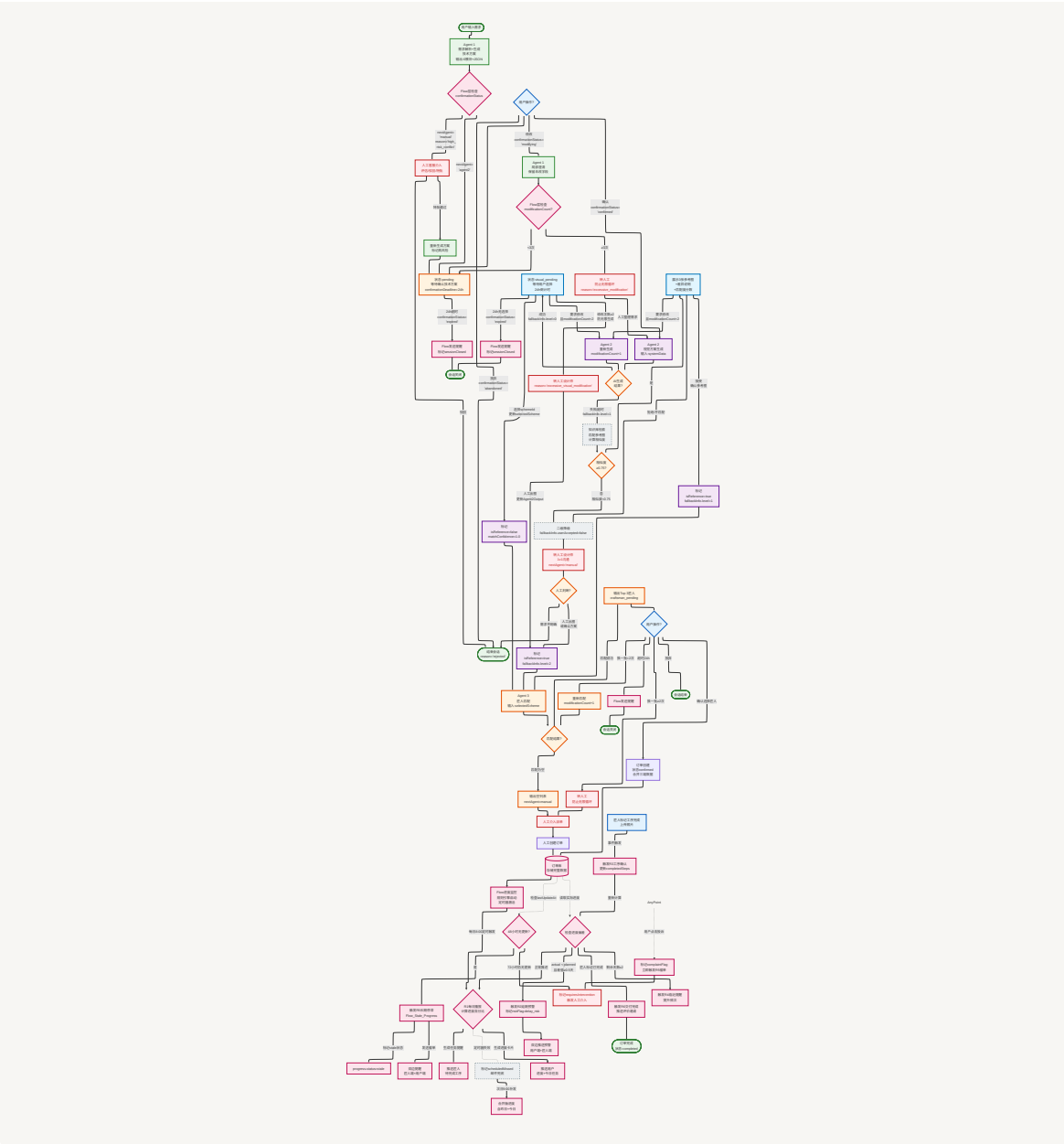
%% 定时失效兜底
Flow_Daily -->|定时器失效| Fallback_Schedule[标记scheduledMissed<br/>邮件兜底]:::fa
llback
Fallback_Schedule -->|次日9:00补发| Email_Fallback[合并版进度<br/>含昨日+今日]:::flow

%% 交付完成
Check_Progress -->|匠人标记已完成| R6_Delivery[触发R6交付完成<br/>推送评价邀请]:::flow
R6_Delivery --> END_SUCCESS([订单完成<br/>状态:completed]):::startEnd

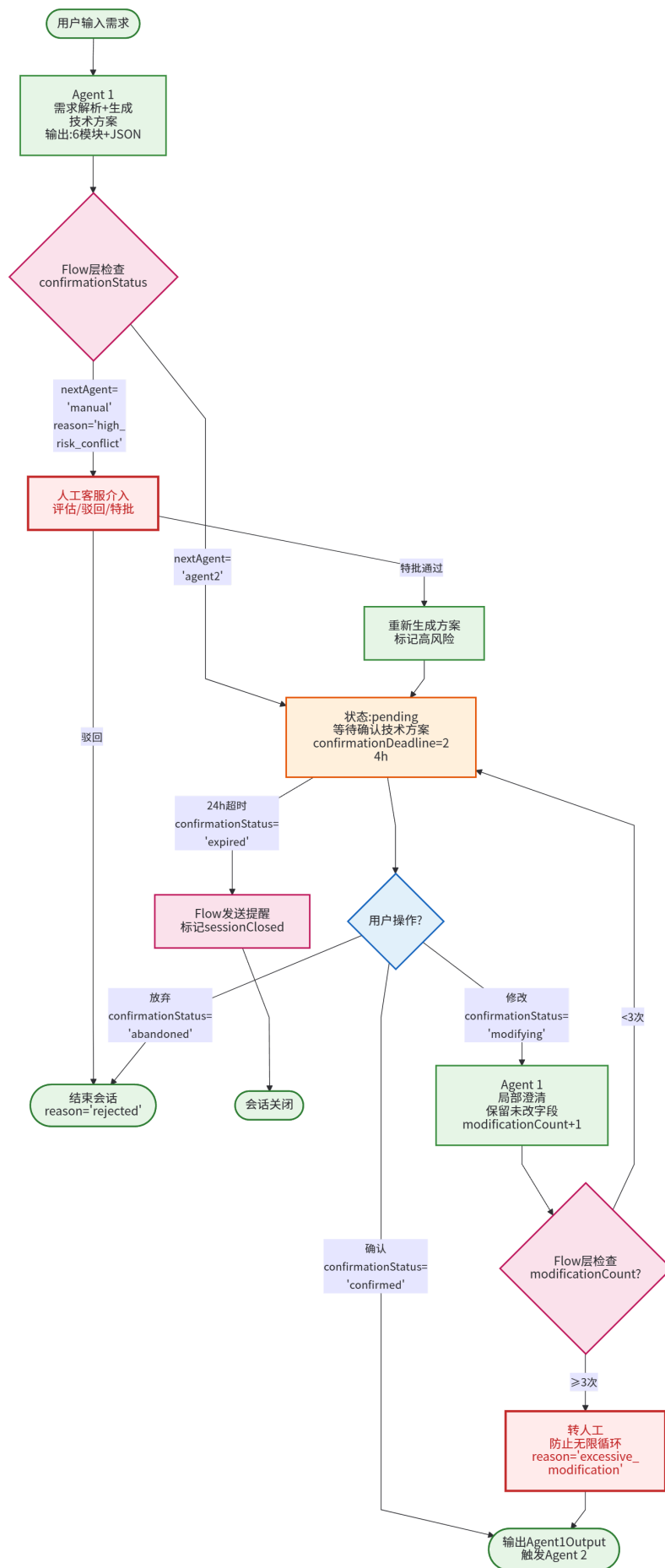
%% 用户投诉贯穿全程
AnyPoint -->|用户点击投诉| Complaint_Flow[标记complaintFlag<br/>立即触发R5催单]:::fl
ow
Complaint_Flow --> Require_Intervention

%% ===== 样式补充 =====
style AnyPoint fill:none,stroke:none
style A3_Prep fill:#f3e5f5,stroke:#6a1b9a,stroke-width:1px
style A3_Prep_Ref fill:#f3e5f5,stroke:#6a1b9a,stroke-width:1px
style A3_Prep_Manual fill:#f3e5f5,stroke:#6a1b9a,stroke-width:1px
style A2_Modify fill:#f3e5f5,stroke:#6a1b9a,stroke-width:1px
style Manual_A1 fill:#ffebee,stroke:#c62828,stroke-width:2px
style Manual_A2 fill:#ffebee,stroke:#c62828,stroke-width:2px
style Flow_Daily fill:#fce4ec,stroke:#c2185b,stroke-width:2px
style Check_Progress fill:#fce4ec,stroke:#c2185b,stroke-width:2px
style Check_Stale fill:#fce4ec,stroke:#c2185b,stroke-width:2px
style R2_Delay fill:#fce4ec,stroke:#c2185b,stroke-width:1px
style R3_Complete fill:#fce4ec,stroke:#c2185b,stroke-width:1px
style R4_Urgent fill:#fce4ec,stroke:#c2185b,stroke-width:1px
style R5_Stale fill:#fce4ec,stroke:#c2185b,stroke-width:1px
style R6_Delivery fill:#fce4ec,stroke:#c2185b,stroke-width:1px

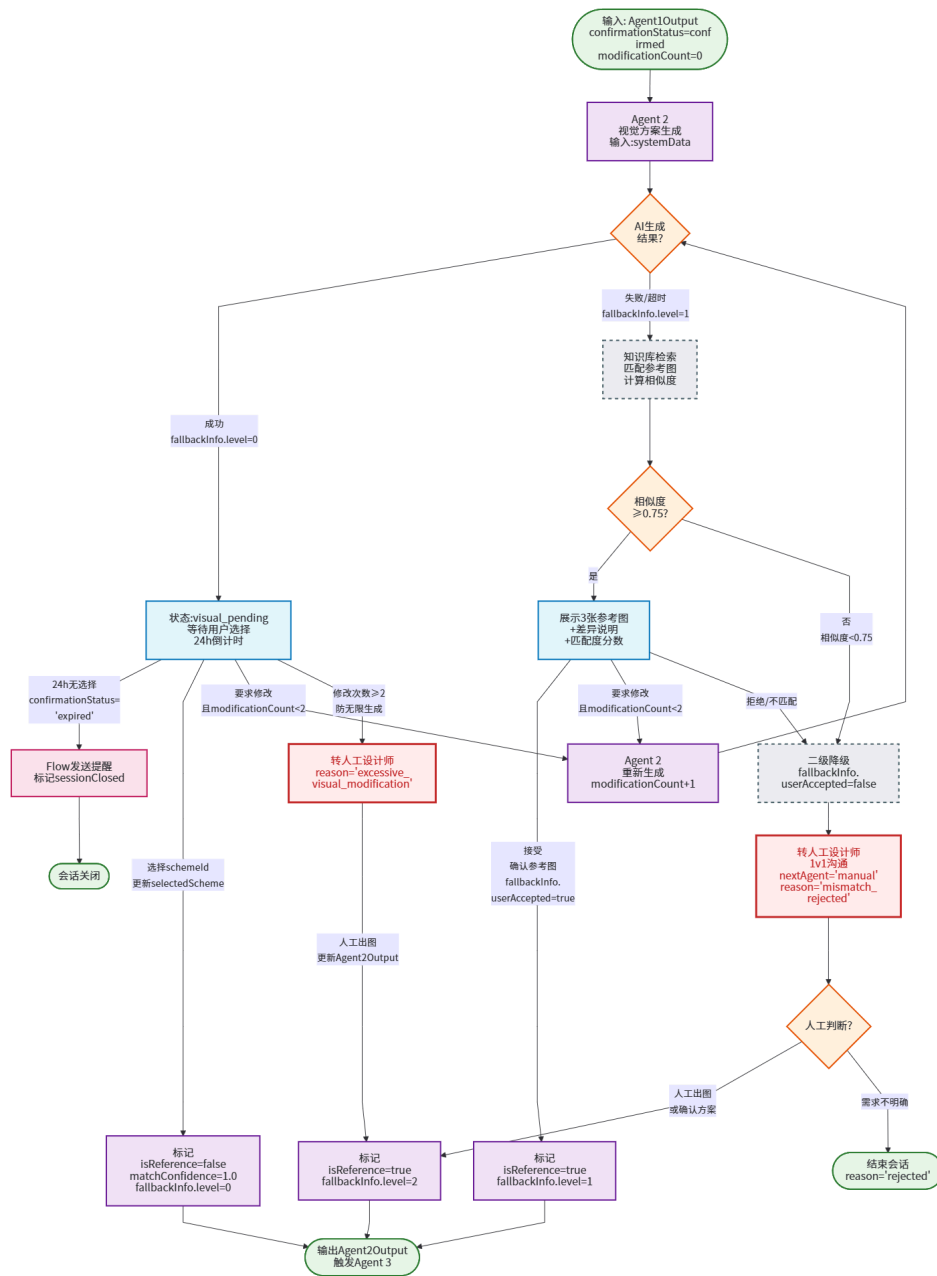
```



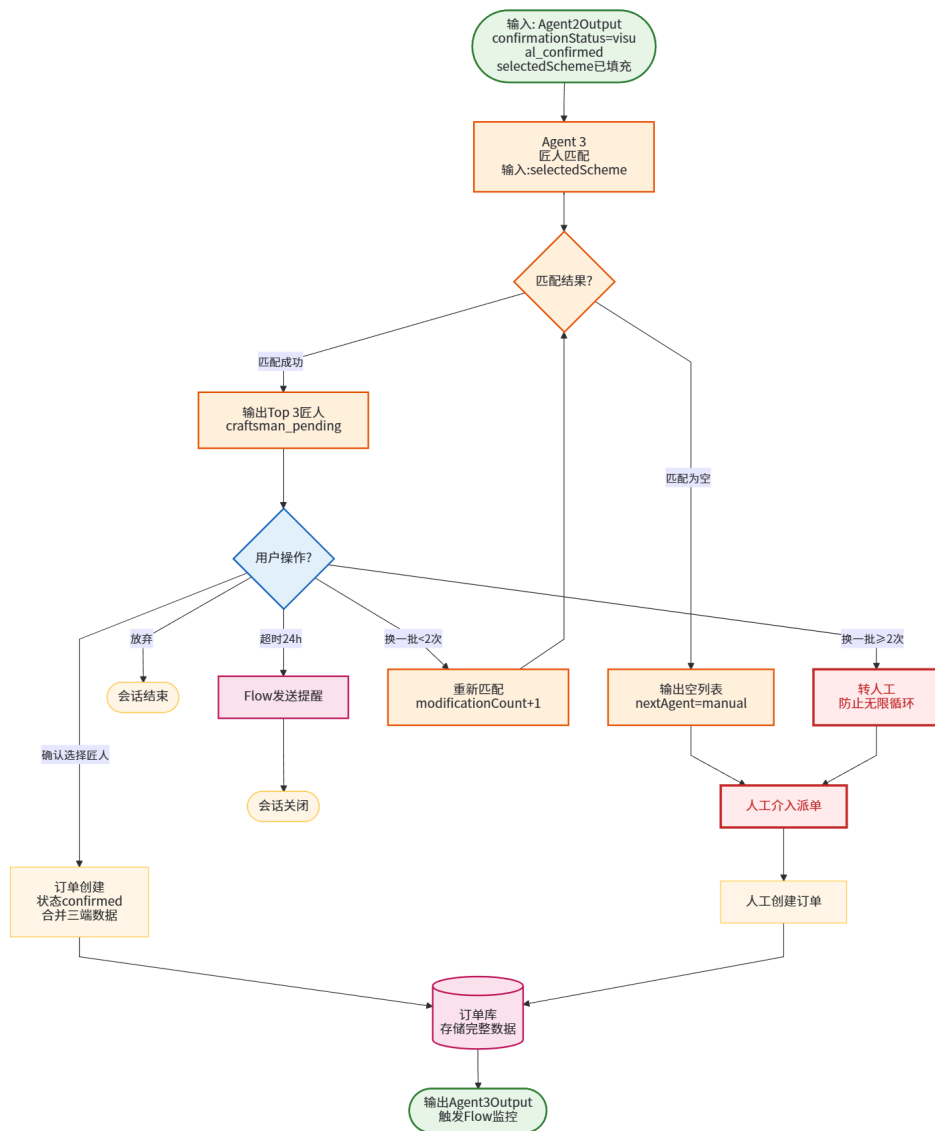
局部工作流程 1: Agent 1 需求解析流程



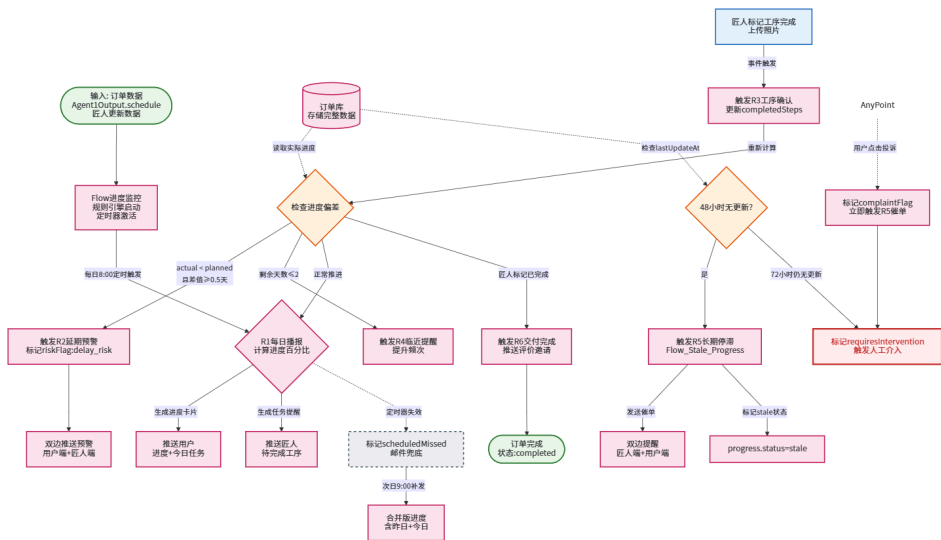
局部工作流程 2：Agent 2 图案生成流程



局部工作流程 3：Agent 3 匠人匹配流程



局部工作流程 4：Flow 进度监控流程



第3章 Agent详细设计

3.1 Agent 1：需求解析Agent

- 产品定位：连接用户需求与匠人执行的桥梁，承担"需求标准化"职责
- 技术架构：Prompt + 知识库RAG混合架构
 - **Prompt层**：负责逻辑推理（完整度判断、澄清策略、流程控制、输出格式化）
 - **知识库层**：6大规则表（图案类型库、风格映射表、载体尺寸规则、预算工期推导、用途/对象推导规则）
 - **架构演进**：V2由单体Prompt解耦为混合架构，Token消耗降低60%，规则可独立维护
 - 迭代决策简要记录：
 - V1：可行性验证 → 基础Prompt版本，包含了 角色定义，核心工作流程，六大推导规则库，澄清策略和示例
 - V1→V2：Coze平台适配，Token超限→prompt内容简要修改,使用Coze的知识库（架构解耦）→降低Prompt长度,Token消耗降低
 - V2→V3：预算工期推导规则不明确→**建立预算-复杂度-工期三角校验表**(业务规则产品化决策/输出格式/边界处理)→推导准确率提升至90%
 - V3→V4：多轮对话记忆丢失→**新增【新增关键记忆与思考机制】章节**(技术架构调整决策)→状态显式化，双方明确信息，减少不必要的思考
 - v4→V5：社交冲突未识别→增加**【Step 2.5冲突预检】**流程（风险拦截）→社交冲突测试综合通过率85%
- 核心流程：意图分析（7维度完整度评分）→澄清对话（P0→P1→P2优先级）→方案生成（双格式输出）
- 状态管理：Agent 1为无状态设计，每次调用独立生成方案。会话状态（confirmationStatus、modificationCount等）由全局Flow维护。
- 输出：
 1. **用户展示**：6模块文字方案（基础参数/构图布局/色彩配置/针法执行/丝线处理/工期安排），每点≤20字，不解释寓意
 2. **系统数据**：结构化JSON（含图案/类型/配景/风格/流派/载体/尺寸/Pantone色号/复杂度/针法列表/预算等级/工期天数/用途/对象）等信息，供下游Agent调用
- 知识库设计：6大规则表（图案类型库、风格映射表、载体尺寸规则、预算工期推导、用途推导规则，对象推导规则）
- 边界处理：技术不可能（照片级写实）→解释限制+替代方案；约束冲突（低成本+高复杂度）→提供折中选项；社交冲突（对象和图案/风格冲突）→Step 2.5预检拦截+建议替代方案；完全无关→礼貌引导回刺绣话题
- 验收标准（V5基线）：
 - 可处理率：≥90%（任意输入能给出方案或合理拒绝）
 - 澄清轮次：≤4轮（80%案例1-2轮完成）
 - 记忆保持：跨轮次信息保持率100%（D3测试5/5分）
 - 风险拦截：社交冲突识别率100%（C3测试5/5分）
 - 综合评分：4.3/5分，核心能力达标（边界拦截5/5、澄清策略4.9/5、记忆保持5/5）

3.2 Agent 2：图案生成Agent

- 产品定位：必须输出**可直接查看的图片+结构化描述**，解决用户"必须看图才能决策"的痛点，承担**视觉信任建立者**职责。
- 输入：来源于Agent 1输出的systemData（图案+风格+复杂度等）
 - 仅当 `Flow`输出状态满足 ``nextAgent: 'agent2'`` 且 ``confirmationStatus: 'confirmed'`` 且 ``modificationCount ≤ 3`` 时触发
- 技术方案：**Prompt工程 + 图像生成API**
 - **AI图像生成**：
 - **首选**：Coze图像生成节点（文生图，支持中文Prompt）

- **备选**：外部API（Midjourney API/Stable Diffusion API，通过HTTP节点调用）
- **图片规格标准**（技术实现要求）：
 - **主图**：1024×1024px，JPG格式，质量90%，单张≤2MB
 - **缩略图**：300×300px，JPG格式，质量80%，用于列表展示
 - **存储方式**：Coze对象存储URL，有效期7天（Agent 3需在7天内完成匹配，超期需重新生成）
 - **命名规范**：`{sessionId}_{schemeId}_{timestamp}.jpg`（如 `sess_abc123_S001_1709876543.jpg`）
 - **格式转换**：若使用Midjourney API（返回PNG），需自动转码为JPG并压缩
- **Prompt构建策略**：`[流派]风格的[图案]刺绣效果图，[复杂度]复杂度，[载体]尺寸，主色调[主色]，[特殊要求]，[用途]场景，白底展示，高清细节，针法纹理清晰，专业产品摄影风格，8k分辨率`
- **输出**：
 - 单次生成3张不同**构图变体**（中心构图/对角构图/留白构图）
 - 每张图附带**差异化描述**
- **异常处理**：
 - AI生成失败（超时/内容违规/算力不足）或 `fallbackInfo.level: 1` 降级为“参考图库匹配”

Coze配置方案：

1. 创建**图像类知识库**（Knowledge），上传历史案例图
2. 设置元数据标签：`pattern`（图案）、`style`（风格）、`complexity`（复杂度）、`carrier`（载体）
3. 使用**图片理解能力**进行向量化检索

检索逻辑：

- 按 `pattern + style + complexity` 三维匹配
- 相似度阈值：**0.75**（低于此值触发二级降级）
- 返回Top 3参考图，附带 `matchConfidence` 分数+差异说明
- 参考图不匹配（相似度<0.75或用户拒绝）或 `fallbackInfo.level: 2`

处理方案：

- 返回通用**流派风格示意图**（如“苏绣淡雅风格参考”）
- 标记 `nextAgent: 'manual'`，强制转人工
- 文案提示：“您的需求较为独特，已为您预约专业设计师1v1沟通”

▼ 与上下游协作机制

- **与Agent 1衔接**
 - **输入依赖**：严格校验 `confirmationStatus: 'confirmed'`，未确认或修改中超限（≥3次）不触发
 - **数据复用**：Agent 1的 `systemData` 直接透传，Agent 2不修改业务字段，仅增加视觉相关字段
- **与Agent 3衔接**
 - **状态依赖**：必须等待用户选择 `selectedSchemeId` 后，才触发Agent 3
 - **数据传递**：
 - Agent 2的输出包含3张待选图片（`visualSchemes`），但**不包含用户选择结果**（`selectedScheme`为null）。完整时序如下：
 1. **初始输出**：Agent 2生成3张图 → 输出JSON（`selectedScheme=null, confirmationStatus=visual_pending`）
 2. **用户交互层（Flow或前端）**：展示3张图，用户点击选择（如选择S001）
 3. **状态更新**：Flow层更新Agent2Output：
 - `workflowControl.selectedSchemeId: "S001"`
 - `workflowControl.confirmationStatus: "visual_confirmed"`

- `systemData.selectedScheme: {填充S001的完整数据}`
- 4. 触发Agent 3: 携带已填充的 `selectedScheme` 触发Agent 3
 - 异常时序:
 - 一级降级 (参考图): 用户点击"接受此风格" → 同上流程
 - 一级降级 (参考图): 用户点击"拒绝/需要更精确" → 触发二级降级 (`fallbackInfo.level=2, nextAgent=manual`)
 - 用户放弃: `confirmationStatus=abandoned` , 流程结束
 - 风险提示: 若 `isReference: true` 且 `matchConfidence < 0.9` , Agent 3需在匠人端显示"此为参考图, 需与用户确认细节"的警示
- 一级降级 (参考图) 时, 用户点击"接受此风格"后:
 1. `fallbackInfo.userAccepted: true`
 2. `workflowControl.confirmationStatus: 'visual_confirmed'`
 3. 触发Agent 3 (与主路径相同)
- 用户点击"拒绝/需要更精确":
 1. `fallbackInfo.userAccepted: false`
 2. 触发二级降级 (`fallbackInfo.level: 2`)
- 验收标准 (AC)

验收项	标准	验证方式
AC-Agent2-1	AI生成成功率≥80%	压测50次生成请求, 统计成功率
AC-Agent2-2	必须返回3张视觉方案图 (AI生成或参考图)	检查输出JSON的 <code>visualSchemes</code> 数组长度
AC-Agent2-3	参考图必须显示匹配度分数, 低于0.75自动触发二级降级	测试用例: 输入冷门图案"独角兽", 验证是否触发人工介入
AC-Agent2-4	用户选择图片后, <code>selectedSchemeId</code> 和 <code>isReference</code> 字段必须正确填充	模拟用户选择方案B, 检查Agent 3接收数据
AC-Agent2-5	视觉方案修改次数≤2次, 超过则强制转人工	模拟连续3次要求修改, 验证是否触发nextAgent: manual且 reason: excessive_visual_modification

- 优化项 (可能未来迭代)
 - 用户上传参考图: 支持用户上传"想要类似这种风格"的图片, Agent 2进行风格迁移而非纯文生图

3.3 Agent 3: 匹配推荐Agent

- 产品定位: 基于 **RAG + Prompt 混合架构** 的智能供需匹配引擎, 负责将已确认的技术方案与平台匠人资源进行精准匹配, 解决"谁能做 + 谁适合"的决策问题, 输出 Top 3 优先推荐列表供用户选择。
- 输入: Agent 1输出的systemData (技术方案数据: 图案+风格+复杂度等) +Agent 2输出后的用户选择后的更新版本的systemData的视觉数据
 - 仅在 **Agent 2 状态更新后的完整输出** (即 `confirmationStatus: 'visual_confirmed'` 且 `selectedSchemeId` 已填充的状态) 后触发
- 技术方案: **RAG知识库+ Prompt 混合架构**
 - **RAG 层: 数据检索**
 - 知识库结构设计
 - 匠人信息表:

字段	类型	检索权重	说明
匠人编号	文本	-	唯一标识
姓名	文本	-	展示名称
擅长流派	数组	高	如["苏绣","现代绣"], 支持多标签

字段	类型	检索权重	说明
可接复杂度	枚举	必填	低/中/高（必须≥订单复杂度）
价格区间	对象	中	{最低价:300, 最高价:5000, 单位:"元/件"}
档期可用日	日期	必填	格式: YYYY-MM-DD
当前负荷	整数	-	在手订单数, 影响排序
作品标签	数组	低	如["擅长花卉","精细满绣"]
历史评分	小数	排序	0-5.0分
接单策略	枚举	排序	质量优先（重技艺匹配）/ 档期优先（重交付速度）

- 检索策略：硬性过滤 + 软性匹配的双层策略
 - 硬性过滤
 - 必须同时满足以下三个条件，否则直接排除：
 - 复杂度门槛：匠人可接复杂度 ≥ 订单复杂度（高难度订单不能派给只做简单活的匠人）
 - 档期可行性：匠人档期可用日 + 制作周期 ≤ 订单要求交付日期（确保能在 deadline 前开工）
 - 预算重叠：匠人最低价 ≤ 订单预算上限（避免推荐连起步价都超预算的匠人）
 - 向量相似度检索
 - 查询条件拼接：将"图案风格 + 刺绣流派 + 特殊工艺"组合成检索语句
 - 匹配维度：流派匹配度 + 风格相似度 + 标签重合度
 - 标签重合度说明
 - 匠人作品标签：如 ["擅长花卉", "精细满绣", "工笔风格", "大面积留白"]
 - 订单需求标签：由Agent 1从用户描述中提取，如 ["花卉", "精细工艺", "淡雅留白"]
 - 重合度计算：交集数量 / 并集数量（如4个标签中重合3个，重合度75%）
 - Top K：召回 10 位候选匠人（供排序层筛选）
 - 推荐理由生成逻辑
 - 每个推荐结果必须包含具体的匹配维度说明，示例：
 - "擅长苏绣与淡雅风格，档期空闲，预算匹配"
 - "技艺精湛（评分4.9），当前负荷低，可优先排单"
 - "擅长复杂满绣（高难度订单），价格区间吻合"
- Prompt 层：推理决策
 - 输入：
 - RAG 返回：Top 10 匠人原始数据（含流派、评分、档期、价格、标签、相似度分数）
 - 订单需求：来自 Agent 1 的 systemData（复杂度、预算等级、工期、流派等）
 - 用户选择：来自 Agent 2 的 selectedScheme（视觉方案确认后触发）
 - 核心推理任务

任务	Prompt 逻辑	输出示例
五维度评分	流派匹配 30 分（完全/相似/不匹配）；复杂度适配 20 分（基础分 + 降维打击加分）；预算匹配 20 分（重合度百分比）；策略契合 20 分（质量派看标签匹配度，档期派看负荷倒数）；历史口碑 10 分（评分比例折算）	<pre>"breakdown": {"genreMatch": 30, "complexityFit": 25, ...}</pre>
策略识别	识别匠人 priorityType，动态调整权重	质量派：标签匹配度权重 ↑；档期派：负荷倒数权重 ↑
推荐理由生成	基于最高得分维度，生成自然语言	<pre>"recommendationReason": "擅长苏绣与淡雅风格，档期空闲，预算匹配"</pre>
风险标记	判断档期冲突、预算紧张、风格冲突	<pre>"riskFlags": ["档期冲突"]</pre>
异常处理	候选 < 3 人时决策降级或转人工	<pre>nextAgent: 'manual'</pre>

- 排序算法（五维度匹配评分机制）

评分维度	分值	计算逻辑
流派精准匹配	30分	完全匹配流派得30分；匹配相似流派（如淡雅风格→苏绣）得15分
复杂度适配	20分	基础分20分；匠人技能高于订单需求（如高手做简单活）额外加分，体现"降维打击"优势
预算匹配度	20分	价格区间重合度越高得分越高
匠人偏好策略	20分	质量派匠人：擅长类型匹配度越高得分越高；档期派匠人：当前在手订单越少得分越高
历史口碑	10分	按历史评分比例折算（4.5分=9分）

▼ Prompt 框架



角色：刺绣匠人匹配专家

输入数据：

- 订单需求：{图案, 风格, 复杂度, 预算等级, 工期天数, 特殊要求}
- 候选匠人（RAG返回）：[{id, name, expertiseGenres, qualityScore, priorityType, priceRange, availability, tags, vectorScore}, ...]

执行步骤：

1. 【硬性过滤校验】确认所有候选已通过复杂度/档期/预算过滤
2. 【五维度评分】对每位候选人计算：
 - 流派匹配（30分）：完全匹配30，相似流派15，不匹配0
 - 复杂度适配（20分）：基础20，技能溢出（高手做简单活）加5分
 - 预算匹配（20分）：价格重合度百分比 × 20
 - 策略契合（20分）：质量派=标签重合度×20，档期派=(1-负荷/10)×20
 - 历史口碑（10分）：评分/5 × 10
3. 【排序】按总分降序，取Top 3
4. 【生成理由】基于最高得分维度，生成一句话推荐理由
5. 【风险标记】检查档期冲突、预算紧张、风格冲突
6. 【输出】严格按Agent3Output JSON格式返回

约束：

- 必须体现质量派vs档期派的排序差异
- 推荐理由必须包含至少两个具体匹配维度
- 风险标记不为空时，需在reason中说明

▼ 技术选型理由

- 为何使用检索增强和纯提示词？
 - 匠人数据动态变化（档期、评分、作品标签经常更新），知识库可独立维护，无需频繁修改提示词
 - 匹配规则复杂（多维度硬性条件），纯提示词难以稳定处理刚性筛选逻辑
 - 需要支持模糊语义匹配（如"水墨风"→"擅长山水/淡雅"），向量检索比关键词匹配更精准
 - 匹配涉及五维度加权计算和策略判断（质量派 vs 档期派），这是 RAG 无法独立完成、必须由 Prompt 层处理的推理任务
- 为何不使用微调模型？
 - 初期匠人数据量小（不足100人），不满足微调所需数据量
 - 匹配逻辑需要可解释性（需向用户说明为何推荐这位匠人），检索增强可追溯匹配依据，微调模型为黑盒
- 为什么采用"硬性过滤 + 软性匹配"双层策略？

策略层级	设计原因	业务价值
硬性过滤	基于访谈中匠人"明确拒单"的底线要求（高难度订单不能接给低技能匠人、档期完全冲突无法交付、预算差距过大直接拒绝）	避免推荐根本不可能成交的匠人，减少无效匹配和匠人拒单率
软性匹配	在硬性条件满足的基础上，解决"风格契合度"问题（如"水墨风"不等于字面出现"水墨"二字，需语义关联）	提升用户满意度，确保推荐匠人不仅"能做"而且"擅长做这种风格"

- 为什么硬性过滤必须同时满足三个条件？
 - 基于匠人拒单的四大场景（访谈记录），任一条件不满足都导致订单无法承接：
 - 复杂度门槛：若匠人只能做"低复杂度"（如简单文字），接到"高复杂度"（如满绣龙凤）订单会直接拒单（"低价糟蹋手艺"或"技术做不到"）
 - 档期可行性：若匠人档期已排到3个月后，用户要求7天交付，时间物理上不可行
 - 预算重叠：若匠人起步价1500元，用户预算仅300元，价格底线无法达成交易
 - 硬性过滤采用"一票否决"制，任一条件不满足直接排除，避免将明显不匹配的匠人送入后续排序，浪费计算资源且损害用户体验。
- 为什么将"图案风格 + 刺绣流派 + 特殊工艺"组合检索，而不用其他维度？
 - 基于语义模糊性和跨维度关联需求：
 - 单维度检索的局限：仅搜"淡雅"可能遗漏标注为"水墨风"但实际风格契合的匠人；仅搜"苏绣"可能错过擅长"现代绣"但能做淡雅风格的高手
 - 组合语义增强："新中式水墨+苏绣+金线点缀"组合后，向量模型能理解这是"需要精细工艺的传统风格"，而非单独关键词的简单叠加
 - 意图完整性：用户描述的"氛围词"（如"仙气"）需要结合流派（苏绣擅长表现）和工艺（细丝线）才能准确映射到匠人能力
 - 语义模糊匹配 vs 精确条件判断：软性匹配解决"风格像不像"的问题，硬性过滤解决"能不能做"的问题。

维度类型	检索方式	原因
图案风格、刺绣流派、特殊工艺	向量相似度检索	这些是描述性文本，存在语义模糊和关联扩展需求（如"水墨风"≈"淡雅"≈"留白多"）
复杂度、工期、预算	硬性过滤	这些是精确数值/枚举，只有"满足/不满足"两种状态，无需模糊匹配

- 输出：
 - 成功匹配（有合格匠人）
返回前三名匠人，包含：排名、匠人编号、姓名、匹配得分、推荐理由、预估报价区间、档期状态、历史评分。
 - 降级匹配（无完美匹配）
匹配状态标记为"部分匹配"，仍返回三位次优选项，并提示："当前无档期完全匹配的匠人，已为您推荐相似人选，或可申请人工客服协调"。
 - 匹配失败（无候选人）
返回空列表，并提供降级方案："暂无可接此刺绣的空闲匠人，请等待人工"。
- 异常处理与边界情况

异常/边界情况	处理策略	说明
检索超时/为空	仅用硬性过滤条件（复杂度+档期+预算）筛选匠人，跳过硬性相似度计算	
匹配为空	返回空列表，全局 workflow 标记 <code>nextAgent: 'manual'</code> ，提示"当前无匹配匠人，申请人工客服协调档期/调整需求" <code>matchingStatus: 'failed'</code>	触发条件：硬性过滤后候选人 < 1 位，或用户连续拒绝 Top 3 推荐（全局 workflow 控制）
档期冲突	标记"需等待X天后开工"，推荐理由改为"技艺匹配，需协调档期"	

异常/边界情况	处理策略	说明
预算紧张	如果匠人最低价接近预算上限，标记"预算偏紧"，提醒用户	
风格冲突	用户要"淡雅"（苏绣特征）但指定"粤绣"匠人时，提示风格不匹配风险	

▼ 与上下游协作机制

• 与 Agent 1/2 衔接（上游）

协作维度	具体机制	关键字段
触发条件	必须等待 Agent 2 状态更新为 <code>confirmationStatus: 'visual_confirmed'</code> 且 <code>selectedSchemeId</code> 已填充（非空）后才触发 Agent 3	<code>workflowControl.confirmationStatus</code> <code>workflowControl.selectedSchemeId</code>
输入依赖	Agent 1 的 <code>systemData</code> 业务数据（复杂度、预算等级、工期、刺绣流派等）由 Flow 层合并传入 Agent 3，用于硬性过滤条件。注：Agent 2 的 <code>systemData</code> 不包含 Agent 1 数据，需由工作流上下文独立提供	图案\图案类型\配景\风格\流派\载体\复杂度\针法列表\丝线规格\预算\预算等级\工期天数\工期类型\工期安排\特殊要求....
	Agent 2 的 <code>systemData.selectedScheme</code> 是核心输入，包含用户最终确认的视觉方案（图案风格、效果图链接、设计说明等）	<code>systemData.selectedScheme.schemeId</code> <code>systemData.selectedScheme.imageUrl</code> <code>systemData.designNotes</code>
异常时序	若 Agent 2 为降级匹配（用户接受参考图），Agent 3 需识别 <code>isReference: true</code> 并在输出中标记 <code>requireManualApproval: true</code> ，提醒匠人"此为参考图需二次确认"	<code>systemData.selectedScheme.isReference</code> <code>workflowControl.requireManualApproval</code>

• 与下游衔接（Flow）

协作维度	具体机制
架构边界	Agent 3 为最后一个实时 Agent，输出后 <code>nextAgent</code> 为 <code>null</code> ，实时链路结束
数据沉淀	用户选择匠人后，前端将匠人信息（来自 Agent 3）+ 视觉方案（来自 Agent 2）+ 业务参数（来自 Agent 1）一并写入订单表
Flow 触发	Flow 定时轮询订单表，读取合并后的完整订单数据（含匠人、方案、工期），推送进度通知

• 验收标准

验收项	标准	验证方式
AC-Agent3-1	高难度订单必须匹配到可接高复杂度的匠人（准确率 100%）	构造高复杂度订单，检查推荐结果
AC-Agent3-2	前三推荐中至少包含一位档期完全合适的匠人	检查 <code>availability.canDeliverBy</code> ≤ 订单 deadline
AC-Agent3-3	每条推荐理由必须包含至少两个具体匹配维度	正则检查推荐理由文本
AC-Agent3-4	排序结果需体现质量派与档期派的差异特征	对比同订单不同策略匠人的排序变化
AC-Agent3-5	RAG 检索失败时自动降级为规则匹配	模拟检索超时，验证输出结果

3.4 Flow：进度监控规则引擎

- 产品定位：基于规则的状态机引擎，承担"全程可视化追踪者"职责。将Agent 1生成的工期计划与匠人实际执行进度进行实时比对，自动生成每日进度推送文案，解决用户"进度失明焦虑"和匠人"被反复催促"的双边痛点。
- 触发方式：
 - 定时触发：定时触发器（每日早 8 点执行）

- **事件触发**：匠人上传工序照片/标记完成某工序时即时触发/用户催促
- **输入**：
 - **计划层**：Agent1Output.systemData.schedule（工期安排：Day1-DayN具体工序）/订单数据信息
 - **执行层**：匠人端更新的实际进度数据（当前工序、完成照片、预计剩余天数）
 - **订单元数据**：订单ID、用户ID、匠人ID、deadline、当前状态（confirmed/production/completed）
- **技术方案：规则引擎（非AI方案）**
 - **选型理由**：进度监控是**确定性状态机**（计划vs实际对比），无需AI不确定性，规则引擎成本低、延迟低。
 - **核心规则库**：

规则ID	规则名称	触发条件	执行动作	输出文案模板
R1	每日进度播报	定时触发（每日8:00）	计算 <code>currentDay/totalDays</code> 得百分比 匹配当日计划工序	"早安！您的刺绣订单已完成60%（3/5天），今日工序：花瓣细节绣制，预计2天后交付"
R2	延期预警	<code>actualProgress < plannedProgress</code> 且差值≥0.5天	标记 <code>riskFlag: delay_risk</code> 双边推送预警	用户端："订单进度略有滞后，匠人正加紧制作，预计延期1天，已为您协调加急" 匠人端："订单【荷花挂画】已落后计划0.5天，请及时更新进度或联系平台"
R3	工序完成确认	匠人标记某工序"已完成"	更新 <code>completedSteps</code> 触发R1重新计算	用户端："🎉好消息！勾线工序已完成，即将进入染色工序"
R4	deadline临近	剩余天数≤2且未完成	提升提醒频次（每日2次） 标记 <code>urgent: true</code>	"🔔 交付倒计时2天，当前进度80%，最后冲刺阶段"
R5	长期停滞	超过48小时无进度更新	触发 R5 催单逻辑（Flow_Stale_Progress）逻辑 触发催单逻辑发送双边提醒	匠人端："订单已 48 小时未更新，请确认进度或联系客服"用户端："您的订单暂未更新，平台已催促匠人"
R6	交付完成	匠人标记"已完成"	触发交付流程 推送评价邀请	"您的刺绣作品已完成！点击查看成品图并确认收货"

- **状态机设计**：
订单生命周期: confirmed → production → [delay_risk] → completed → [review_pending] → closed
production子状态:
 - on_track (正常)
 - slight_delay (滞后<1天, 提醒匠人)
 - severe_delay (滞后≥1天, 双边预警+人工介入)
 - stale (停滞 48h+, 触发 R5 催单)
- **输出**：
 - **给用户**：进度卡片（百分比圆环+当日工序+ETA+成品预览图占位符）
 - **给匠人**：任务提醒（今日待完成工序+剩余工期+用户催单标记）
 - **给系统**：FlowOutput（用于更新订单状态、触发异常处理）
- **与上下游协作机制**
 - **与上游（订单创建）衔接**：
 - **触发前提**：订单状态必须为 `production`（已确认匠人且匠人已接单）
 - **计划基线**：读取 Agent1Output.systemData.schedule 作为对比基准，**不修改** Agent 1 生成的工期安排
 - **与Agent 3衔接（横向）**：
 - **数据复用**：读取Agent 3输出的 `craftsman.availability.canDeliverBy` 作为初始ETA基准

- **状态同步**：当Flow检测到 `severe_delay` 时，更新订单状态为 `exception`，Agent 3侧显示该匠人"有延期订单"
- **与下游（业务系统/人工）衔接**：
 - **架构边界**：Flow为4-Agent架构终点，无 `nextAgent` 字段，输出直接写入订单数据库
 - **人工介入点**：当 `requiresIntervention: true`（如连续 3 天 `severe_delay` 或 R5 停滞若无响应），触发人工客服跟进（`manual` 状态）
 - **用户投诉**：任意环节用户点击"催单/投诉"，Flow立即触发R5规则，生成催单通知并标记 `complaintFlag`
- **异常处理与降级策略**

异常场景	识别方式	处理策略	兜底方案
定时失效/推送失败	定时触发器未按时执行（通过日志监控）	标记 <code>scheduledMissed: true</code>	邮件兜底 ：次日早9:00补发合并版进度（含昨日+今日）
数据不一致	匠人标记完成但 Agent1.schedule无对应工序	报警+人工核对	使用最近有效进度，文案模糊化为"制作进行中"
匠人失联	超过72小时无更新且未请假	标记 <code>craftsman_unresponsive</code> 触发人工介入	平台临时接管，联系备用匠人
匠人长期停滞 (R5)	超过 48 小时无进度更新	触发催单提醒 → 72 小时仍无更新则标记 <code>requiresIntervention</code>	人工客服介入联系匠人

• 验收标准

验收项	标准要求	验证方式
AC-Flow-1	进度计算准确率 100%（计划 vs 实际天数偏差 ≤0.5 天）	对比 10 个订单的手工计算与系统计算
AC-Flow-2	48 小时停滞检测准确率 100%	模拟 48 小时无更新，验证 R5 触发
AC-Flow-3	定时触发成功率 ≥99.5%（1周统计）	监控日志，统计未触发次数/总应触发次数
AC-Flow-4	延期预警提前量 ≥24 小时	模拟滞后订单，验证预警触发时点
AC-Flow-5	定时失效后邮件兜底 100%覆盖	模拟定时器故障，验证邮件补发

第4章 技术方案选型

4.1 方案对比矩阵

维度	Prompt工程	RAG	微调	适用Agent
适用场景	逻辑推理、创意生成	知识检索、规则查询	特定领域精准控制	-
成本	低（按Token计费）	中（向量库+检索成本）	高（训练成本）	-
开发周期	短（数天）	中（1-2周建库）	长（1月+）	-

Agent	任务特性	最终选择	关键决策理由
Agent 1	逻辑推理+规则推导	Prompt+RAG混合混合架构	纯Prompt Token超限且难维护；纯RAG无法处理澄清策略等逻辑推理
Agent 2	创意生成+视觉输出	纯Prompt	RAG限制创意发散；微调数据量不足（<1000样本）；文生图需开放式Prompt
Agent 3	知识检索+精准匹配	Prompt+RAG混合混合架构	匠人数据动态变化，需向量检索语义匹配；五维度排序与推荐理由生成需Prompt推理；微调黑盒不可追溯
Flow	确定性状态流转	规则引擎	进度计算是确定性问题，规则引擎成本低（无Token消耗）、延迟低、可预测

4.2 Agent 1混合架构详解

具体迭代过程请看附录C

- 为何不用纯Prompt：刺绣规则表过大（Token超限），且需频繁更新（图案类型扩展，维护困难），成本高
- 为何不用纯RAG：需求解析需要逻辑推理（完整性判断、澄清策略决策），非简单知识问答

- 技术实现
 - Prompt节点：逻辑推理、流程控制、输出格式化，可以灵活调整对话策略，无需改动规则数据
 - 知识库节点：刺绣领域规则存储与检索，规则可独立维护，Token消耗降低
 - 代码节点：数据格式化、状态管理，隐藏思考过程，输出规范JSON

4.3 Agent 2：为什么坚持纯Prompt而非RAG辅助生成

曾考虑的方案：RAG+生成混合

- 思路：用知识库检索历史案例图，作为生成参考
- 否决原因：
 1. 创意限制：RAG检索的"最相似案例"会束缚生成多样性，用户需要3张**构图变体**（中心/对角/留白），而非相似图
 2. 语义鸿沟：用户描述"温柔一点"无法直接匹配知识库标签，需Agent 1先翻译为技术参数
 3. 成本叠加：RAG检索+图像生成双重调用，成本高于纯生成

降级策略（非主路径RAG）

- 仅当AI生成失败（超时/违规/算力不足）时，才启用知识库检索参考图
- 此时RAG作为**兜底方案**而非**主路径**，不牺牲主路径的创意自由度

4.4 Agent 3：RAG + Prompt 混合

层级	职责	技术实现
RAG 层	硬性过滤 + 候选召回	知识库检索（复杂度/档期/预算过滤）+ 向量相似度检索（流派/风格/标签）
Prompt 层	匹配逻辑推理 + 推荐理由生成 + 风险判断	Coze 提示词节点，输入检索结果，输出排序与解释

RAG的核心优势

需求	RAG解决方案	Prompt工程难点
动态数据	知识库独立维护，实时更新匠人档期/评分	需每日修改Prompt并重新部署
硬性过滤	代码层实现复杂度≥订单、档期可行、预算重叠三条件	Prompt中写条件判断，易出错且难调试
语义匹配	向量检索"水墨风"→"淡雅留白"的语义关联	关键词匹配遗漏同义表达
可解释性	检索结果带匹配标签，为Prompt生成推荐理由提供素材	需额外设计解释逻辑

prompt的优势

任务	RAG 能做的事	Prompt 必须做的事
排序评分	返回 10 个候选匠人及其字段	五维度加权计算总分 (30/20/20/20/10)，处理"质量派 vs 档期派"不同策略
推荐理由	返回匹配标签交集	生成自然语言理由："擅长苏绣与淡雅风格，档期空闲，预算匹配"
风险标记	返回档期/预算原始值	判断"预算偏紧""档期冲突"等风险等级
异常处理	返回空列表或超时	决策：降级规则匹配 / 转人工 / 提示用户调整需求

核心区别

维度	Agent 1	Agent 3
调用顺序	Prompt 调用 RAG 节点获取规则	RAG 先检索，结果输入 Prompt
RAG 作用	查询静态规则表（图案类型、预算推导）	检索动态数据（匠人档期、评分、标签）
Prompt 作用	核心逻辑控制（澄清策略、完整度判断）	后处理推理（排序算法、解释生成）

维度	Agent 1	Agent 3
主导方	Prompt	RAG

4.5 Flow：为什么用规则引擎而非AI方案

AI方案的冗余性

- 进度监控是**确定性计算**：实际进度 = 已完成工序数 / 总工序数，无需AI推理
- 状态判断是**条件分支**：if 滞后≥0.5天 then 触发R2预警，规则明确无歧义
- 引入AI会增加：Token成本、响应延迟、结果不确定性

选型收益

- 成本：无Token消耗，仅计算资源（可忽略）
- 延迟：规则计算<50ms，AI方案通常>500ms
- 可靠性：确定性输出，无幻觉风险

4.6 跨Agent记忆设计

短期记忆（会话级）

- 存储：Redis，Key= `session:{sessionId}`
- TTL：24小时（匹配confirmationDeadline）
- 内容：Agent 1澄清状态、Agent 2选择结果、修改次数计数器

长期记忆（用户级）

- 存储：Vector DB（用户偏好向量）
- 内容：历史订单的图案类型、风格偏好、常用对象（如多次送母亲）
- 用途：跨会话推荐默认选项（"您上次选择了淡雅风格，这次是否延续？"）

跨Agent数据流

Agent 1输出JSON → Redis缓存 → Agent 2读取systemData →
用户选择后更新Redis → Agent 3读取合并数据 →
订单创建后持久化 → Flow定时读取

附录

- **附录A**：[Agent 1需求解析Agent 设计文档](#)

[附录B：Agent间JSON Schema定义](#)

[附录C：Agent 1 迭代决策线（V1–V5）](#)

- **附录D**：[谈话记录](#)

[附录E：成本估价](#)

[数据接口契约](#)